CSCI-473: Applied Data Science Homework Assignment 2
Exploratory and Statistical Analysis of Bike-Sharing Demand

Alhassane Diallo
004001562
alhassanediallo@bison.howard.edu

Introduction:

This report looks at the UCI Bike Sharing dataset, which contains hourly bike rental data from Washington, D.C. for 2011 and 2012. The file bike_sharing_hour.csv has 17,379 rows and 17 columns. It includes information such as the date, hour, season, weather, temperature, humidity, wind speed, and number of casual, registered, and total rentals.

The main goal of this analysis is to understand how bike rentals change depending on the time of day, type of day, season, and weather and to practice working on larger data sets.. I completed the analysis in Python using pandas, NumPy, Matplotlib, and Seaborn. Overall, this assignment helped me practice the steps of exploratory data analysis (EDA) and learn how to understand the results instead of just calculating them.

Task 1:

Report:: Issues or features that require attention before analysis.

- dteday is stored as object (text), not a datetime.
- season, weathersit, yr, workingday and holiday are numeric codes, not labels.
- temp, atemp, hum and windspeed are normalized (0–1), not real units.

What I did:

I loaded the CSV into a DataFrame named df with pd.read_csv() and inspected it with head(), tail(), shape, dtypes, info(), describe(), isna().sum() and duplicated().sum().

Results:

The dataset has 17,379 rows and 17 columns. Every column has 17,379 non-null values, so there are no missing values, and there are 0 exact duplicate rows. Twelve columns are int64, four are float64 (the normalized weather measurements), and one

(dteday) is stored as an object. The records run from 2011-01-01 hour 0 to 2012-12-31 hour.



```
The missing values per column:
[11]: instant       0
dteday        0
season        0
yr            0
mnth          0
hr            0
holiday       0
weekday       0
workingday    0
weathersit    0
temp          0
atemp         0
hum           0
windspeed     0
casual        0
registered    0
cnt           0
dtype: int64

[12]: # 9. the number of exact duplicate rows.
print("The number of duplicated rows:", df.duplicated().sum())

The number of duplicated rows: 0
```

```
[9]: # 6. the output of info();
print("Data file Info:")
df.info()

Data file Info:
<class 'pandas.DataFrame'>
RangeIndex: 17379 entries, 0 to 17378
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   instant     17379 non-null  int64
 1   dteday      17379 non-null  str
 2   season      17379 non-null  int64
 3   yr          17379 non-null  int64
 4   mnth        17379 non-null  int64
 5   hr          17379 non-null  int64
 6   holiday     17379 non-null  int64
 7   weekday     17379 non-null  int64
 8   workingday  17379 non-null  int64
 9   weathersit  17379 non-null  int64
 10  temp        17379 non-null  float64
 11  atemp       17379 non-null  float64
 12  hum         17379 non-null  float64
 13  windspeed   17379 non-null  float64
 14  casual      17379 non-null  int64
 15  registered  17379 non-null  int64
 16  cnt         17379 non-null  int64
dtypes: float64(4), int64(12), str(1)
memory usage: 2.3 MB
```

Task 2:

Report: Describe the renaming, conversion, and category-labeling steps.

What I Did:

Renaming Columns: I made a working copy called clean_df and renamed the columns to give them clearer names. (dteday became date, yr became year_code, weathersit became weather_code, and cnt became total_rentals.)

Data Type Conversion: I converted the date column into datetime format using pd.to_datetime() with errors="coerce". I also created a year column to show the actual year, 2011 or 2012. Then, I converted normalized measurements into more familiar units, such as Celsius for temperature and percentages for humidity.

Category Labeling and Mapping: I used Python dictionaries to map numerical codes to descriptive labels, making the data easier to read.

- Season Mapping: {1: "Spring", 2: "Summer", 3: "Fall", 4: "Winter"}
- Weather Mapping: {1: "Clear/Partly Cloudy", 2: "Mist/Cloudy", 3: "Light Rain/Snow", 4: "Heavy Rain/Snow/Fog"}
- Day Type Mapping: {1: "Working Day", 0: "Nonworking Day"}

These mappings were applied to create the season_label, weather_label, and day_type columns.

Results:

The DataFrame increased from 17 to 25 columns. The columns contain different data types:

- Datetime (datetime64): The date column stores dates in a format pandas can recognize.
- Integers (int64 / int32): Columns such as year, month, hour, and total_rentals contain whole numbers.
- Floats (float64): Temperature, humidity, and wind speed measurements contain decimal values rounded.
- Text (object): The season_label, weather_label, and day_type columns contain descriptive labels.

```
[26]: First five prepared rows (new columns):
```

	year	season_label	weather_label	day_type	temperature_c	apparent_temperature_c	humidity_percent	wind_speed
0	2011	Spring	Clear/Partly Cloudy	Nonworking Day	3.28	3.0014	81.0	0.0
1	2011	Spring	Clear/Partly Cloudy	Nonworking Day	2.34	1.9982	80.0	0.0
2	2011	Spring	Clear/Partly Cloudy	Nonworking Day	2.34	1.9982	80.0	0.0
3	2011	Spring	Clear/Partly Cloudy	Nonworking Day	3.28	3.0014	75.0	0.0
4	2011	Spring	Clear/Partly Cloudy	Nonworking Day	3.28	3.0014	75.0	0.0

Task 3:

Report Requirement

List the rules you validated and explain what each assertion checks. Include a clear screenshot showing the successful validation output.

What I did:

I used Python assert statements to check that the cleaned dataset was accurate and consistent. These checks help catch errors before analyzing the data.

1. Unique Identifier Check: assert clean_df["instant"].is_unique

   Checks that every row has a unique identifier and no identifiers are repeated.

2. Valid Hour Range: assert clean_df["hour"].between(0, 24).all()

Checks that hour values fall between 0 and 24. However, valid hours should be 0 through 23, so the code should use between(0, 23).

3. Valid Month Range: assert clean_df["month"].between(1, 12).all()

Ensures that all month values fall between 1 and 12.

4. Season Label Validation: assert clean_df["season_label"].isin(allowed_seasons).all()

Checks that every season label matches one of the allowed categories: Spring, Summer, Fall, or Winter.

5. Weather Label Validation: assert clean_df["weather_label"].isin(allowed_weathers).all()

Ensures that all weather labels match the approved categories, such as Clear/Partly Cloudy or Mist/Cloudy.

6. Non-Negative Rental Counts: assert (clean_df[["total_rentals", "casual_rentals", "registered_rentals"]] >= 0).all().all()

Checks that rental counts are never negative.

7. Rental Total Consistency: assert (clean_df["total_rentals"] == clean_df["casual_rentals"] + clean_df["registered_rentals"]).all()

Confirms that total rentals equal casual rentals plus registered rentals for every row.

8. Missing Value Check: assert not clean_df[["date", "hour", "season", "weather_label", "temperature_c", "humidity_percent", "wind_speed", "total_rentals"]].isna().any().any()

Ensures that the selected important columns do not contain missing values.

9. Duplicate Row Check: assert not clean_df.duplicated().any()

Checks that there are no completely duplicated rows in the dataset.

Results:

All assertions passed. At first I had some assertion failure but I found the problem to be linked to code logic typically with mapping assertions so I quickly fixed those.

```
Confirmed: Instant contains unique values
Confirmed: All hours fall under 24
Confirmed: All hours fall between 1 - 12
```

```
Seasons codes Verified
Weather Codes Verified
```

```
Confirmed: Rental counts are all postive
Confirmed: Total rentals = casual rentals + registered rentals
Confirmed: No missing values found
Confirmed: No duplicated entries found
```

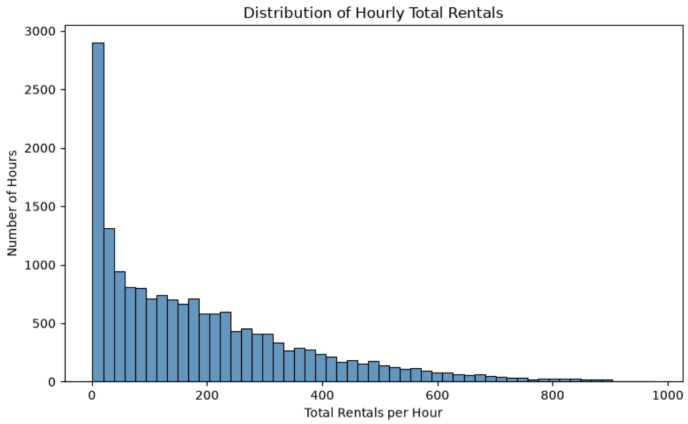
Task 4: Exploratory Data Analysis and Visualization

What I Did

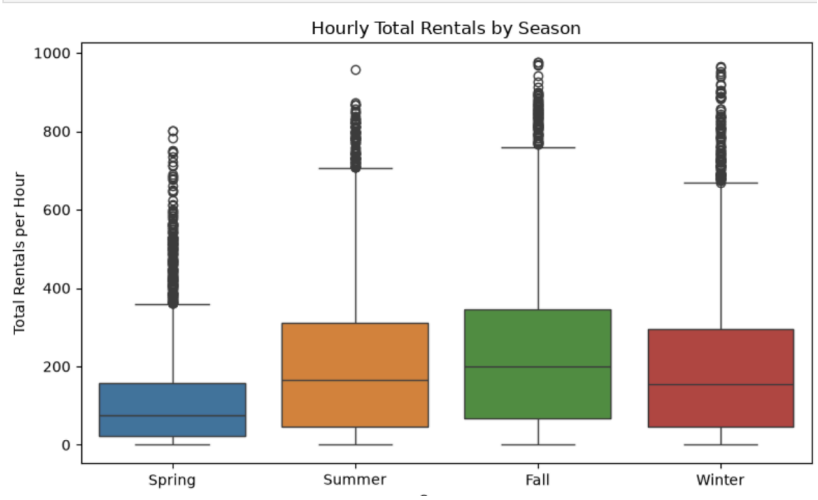
I used Python functions such as agg(), value_counts(), and groupby().agg() to summarize hourly bike rentals. I calculated statistics like the mean, median, standard deviation, minimum, and maximum. I also compared rental counts by season, weather condition, day type, and hour of the day. Finally, I used Seaborn (sns) to create plots to visualize the distributions and compare rental patterns.

Results

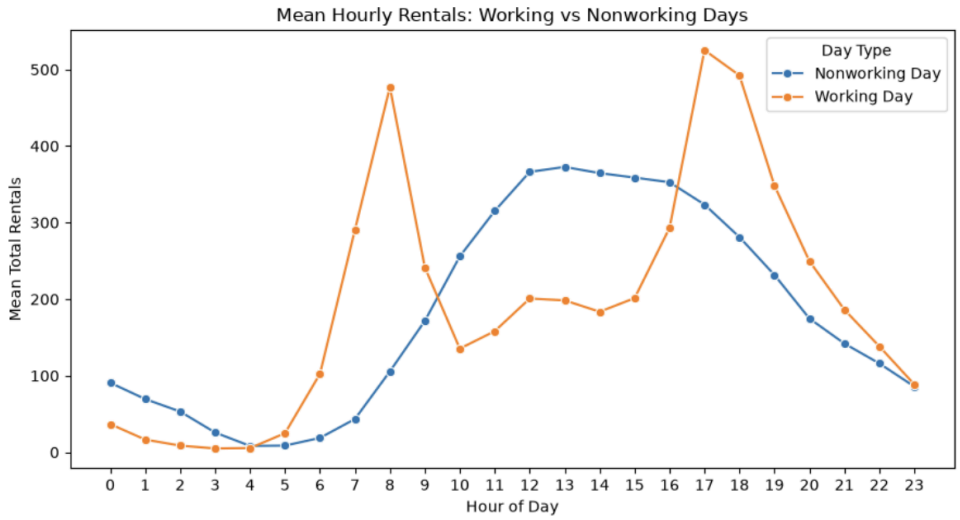
The mean rental count was 189.46, higher than the median of 142, showing that the data is right-skewed. Fall had the highest demand, while Spring had the lowest. Working days had slightly higher average rentals than nonworking days. Demand was lowest at 4 a.m. and peaked at 5 p.m. with 461.45 rentals, suggesting that commuting hours strongly affect demand.



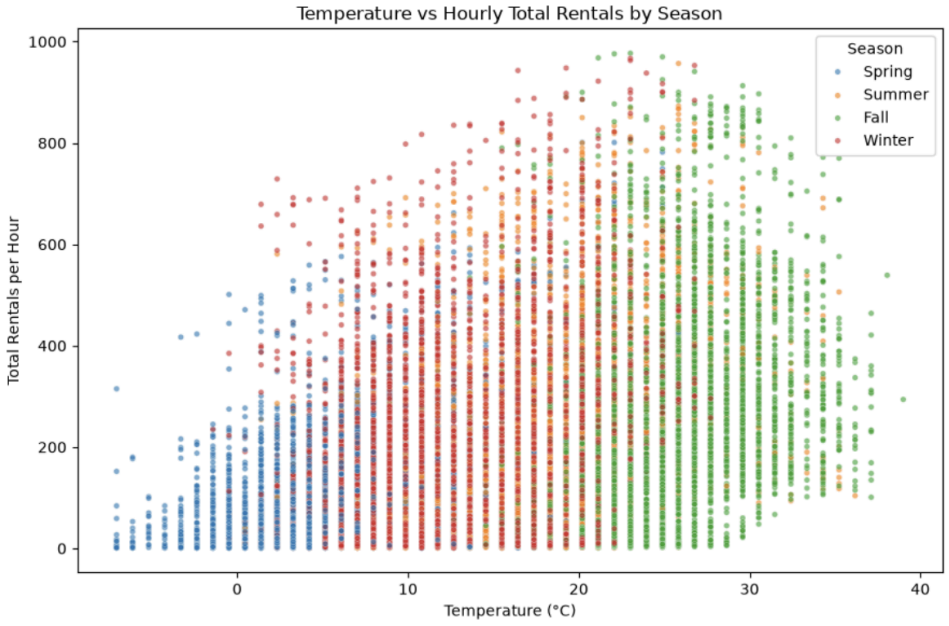
The histogram is strongly right-skewed. Most hours have low rental counts, especially at night and early in the morning, while a smaller number of hours have over 900 rentals. Because of this, the median of 142 rentals better represents a typical hour than the mean of 189.46 rentals. One limitation is that the histogram combines all hours, seasons, and day types, making it harder to see how rental patterns change throughout the day.



Fall has the highest median rental count, while Summer and Winter have similar results. Spring has the lowest median, with fewer than 100 rentals. All seasons have high outliers, which are likely caused by busy commuting hours rather than errors, so I kept them in the dataset.



This graph shows a clear difference between working days and nonworking days. Working days have two peaks, around 8 a.m. (480 rentals) and 5–6 p.m. (520 rentals), which likely reflect commuting. Nonworking days have one broad peak between noon and 4 p.m., with about 350–370 rentals, likely due to leisure riding. The overall daily averages from Part A hide these differences. One limitation is that the graph shows average rentals only, so it does not show how much rentals vary from day to day.



Rentals generally increase as temperature rises. Below about 5°C, very few hours exceed 400 rentals, while the busiest hours occur between 20°C and 32°C. However, the relationship is not

perfectly linear because rentals level off and dip slightly at the hottest temperatures. Season also affects the results. This makes it difficult to separate the effects of temperature and season. Rental counts also vary widely at the same temperature, likely because the time of day affects demand.

Task 5: Probability, Correlation, and Interpretation

A. What I did:

I defined high-demand hours as hours where total rentals were at or above the 75th percentile using quantile(0.75). I stored the results in a Boolean column called high_demand. Then, I used Boolean masks with & to calculate probabilities by dividing the number of rows meeting both conditions by the total number of rows meeting the given condition.

Results:

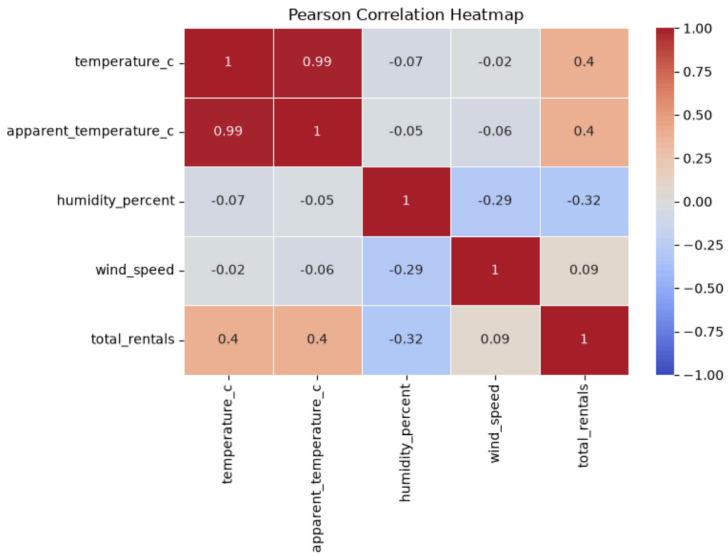
The probability of high demand was 25.1%, slightly above 25% because some hours had exactly 281 rentals. Nonworking days had a slightly higher chance of high demand (26.1%) than working days (24.6%). Clear or partly cloudy weather also increased the chance to 28.3%. This suggests that weather and daily schedules affect rental demand, but these results only describe the 2011–2012 data and do not prove cause and effect.

B.

What i did:

I computed the Pearson correlation matrix with clean_df[cols].corr(method="pearson").round(2) and plotted it with sns.heatmap(annot=True, vmin=-1, vmax=1).

Results



Temperature had the strongest correlation with total rentals (r = 0.40). Apparent temperature had a similar correlation, while humidity had a negative correlation (-0.32) and wind speed had almost no correlation (0.09). However, correlation does not mean causation. Temperature is related to season and time of day, which also affect rental demand. Since the relationship is not purely linear, other factors may explain the increase in rentals. More analysis would be needed to determine whether temperature directly affects demand.

Task 6:

A.

What I did:

Using clean_df.sample(n=500, random_state=42), I drew a simple random sample of 500 rows without replacement.

Result:

The full dataset mean was 189.46 rentals per hour, while the sample mean was 183.92, a difference of 5.54 rentals. This difference is due to sampling variability because the sample includes only 500 of the 17,379 rows. The result is normal and does not mean there is an error in the data.

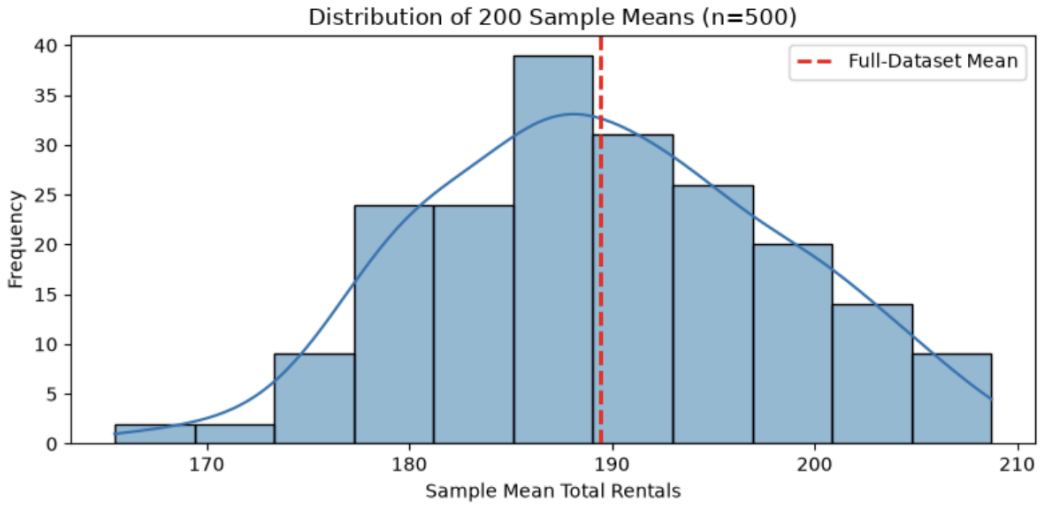
B.

What I Did

I created one random number generator using np.random.default_rng(42) and used it to draw 200 different samples, each containing 500 rows without replacement. This made the experiment reproducible. I calculated the mean for each sample and plotted their distribution in a histogram, with the full-dataset mean shown as a red dashed line.

Results

The average of the 200 sample means was 189.56, very close to the full-dataset mean of 189.46. The standard deviation of the sample means was 8.62, close to the theoretical standard error of about 8.1. The histogram was roughly bell-shaped and centered near the full-data mean, supporting the Central Limit Theorem. This shows that individual samples can differ from the true mean due to sampling variability.



C.

What I did:

I used a random number generator with seed 42 to create 5,000 bootstrap samples, each containing 17,379 rental values sampled with replacement. I calculated the mean for each sample, then used the results to estimate the standard error and a 95% confidence interval.

Results

The bootstrap standard error was 1.37 rentals, close to the theoretical estimate of 1.38. The 95% confidence interval was 186.74 to 192.10 rentals per hour. This interval estimates the uncertainty around the average rental count, not the range of individual hourly rentals.

One limitation is that hourly rentals are time-dependent, but the bootstrap method treats each hour as independent. A block bootstrap may provide a more reliable estimate of uncertainty. These results apply only to this dataset.

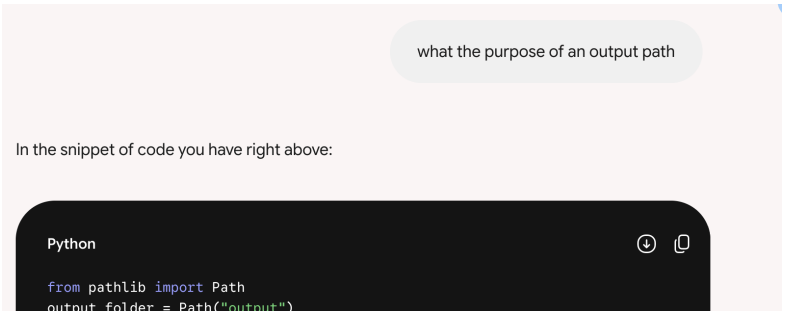
D.

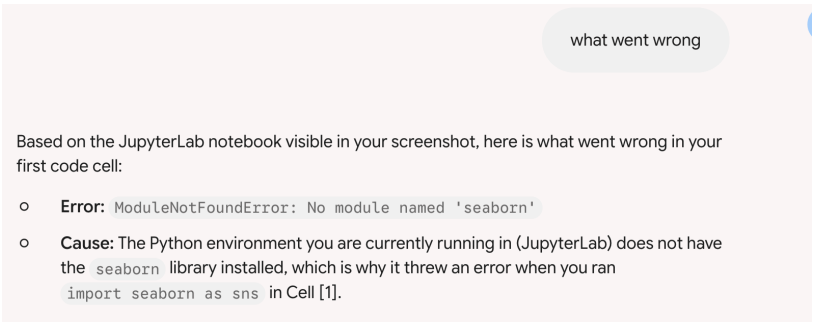
How I Verified the Output File:

I used to_csv() to save the cleaned DataFrame as a CSV file with index=False so the row index would not be included. I used read_csv() to reload the saved file into a new DataFrame called verified_df. I then used head() to check the first five rows, shape to check the number of rows and columns, and isna().sum() to count missing values in each column. Finally, I used assert to confirm that the reloaded dataset's shape matched the original dataset's shape. Since the assertion passed, the export was verified.

Gen Ai Statement:

Gemini was used in this assignment to help understand code and fix errors





4. Analysis Summary

The dataset shows that bike rentals change depending on the time of day, season, and weather. On working days, rentals are highest during morning and evening rush hours, while on nonworking days, rentals are highest around midday. Fall has the highest average demand, and winter has the lowest. The average number of rentals per hour is 189, but the median is 142, showing that some very busy hours raise the average. Warmer temperatures are generally linked to more rentals, while higher humidity is linked to fewer rentals. Overall, time of day and season are important patterns in bike rental demand, although weather alone does not explain why demand changes.

6. Conclusion

This assignment taught me that data analysis is about asking the right questions and checking the data before using it. Cleaning and validating the dataset helped me catch issues that could affect my results. Looking at rentals by hour and day type helped me see the difference between commuting and leisure patterns. I also learned that having a higher average does not always mean there is a higher chance of high demand. The sampling and bootstrap exercises helped me understand how results can change between samples and how confidence intervals estimate the average. Finally, I learned that temperature has a moderate relationship with bike rentals, but this does not mean temperature directly causes higher demand. Other factors, such as the season and time of day, can also affect rentals.